



Universidad Nacional Autónoma de México

Facultad de Ingeniería

Organización y Arquitectura de Computadoras

Proyecto 2. Filtro digital IIR de segundo orden en ensamblador

Grupo 1

Ortega de la Paz Rafael 420054085

19 de octubre de 2026

Objetivo

Diseñar e implementar en lenguaje ensamblador, con la unidad de punto flotante de la Raspberry Pi Pico 2, un filtro digital IIR pasabajas de segundo orden con un tiempo de muestreo de 0.05 s, y visualizar la señal de entrada y la de salida en el Plotter de Thonny.

Introducción

La Raspberry Pi Pico 2 utiliza el microcontrolador RP2350, que integra dos núcleos Arm Cortex-M33 a 150 MHz. A diferencia de la Pico original (RP2040, núcleos Cortex-M0+), cada núcleo Cortex-M33 incluye una unidad de punto flotante (FPU) de precisión simple con 32 registros de 32 bits, S0 a S31, que ejecuta en hardware la suma, la resta, la multiplicación, la división y la raíz cuadrada. La FPU no ofrece funciones trascendentes como seno, coseno o logaritmo; esas deben construirse a partir de las operaciones básicas.

MicroPython permite escribir funciones en ensamblador Thumb dentro de un programa en Python mediante el decorador *@micropython.asm_thumb*. Los primeros cuatro argumentos de la función llegan en los registros r0 a r3 y el valor de regreso se toma de r0. Cuando el argumento es un arreglo (*array*), el registro recibe la dirección de su primer elemento, de modo que el ensamblador puede leer y escribir la memoria del arreglo con instrucciones de carga y almacenamiento (ldr/str para enteros, vldr/vstr para flotantes). Como la memoria se direcciona por bytes y cada dato de 32 bits ocupa cuatro, el desplazamiento del elemento i es $4 \cdot i$.

Un filtro IIR (de respuesta infinita al impulso) calcula cada muestra de salida a partir de las entradas actual y anteriores y de las salidas anteriores. Para segundo orden, su ecuación en diferencias es:

$$y[k] = a_0x[k] + a_1x[k-1] + a_2x[k-2] - b_1y[k-1] - b_2y[k-2]$$

El filtro se diseñó a partir del prototipo analógico pasabajas de segundo orden, con frecuencia natural ω_0 y factor de amortiguamiento ζ:

$$H(s) = \omega_0^2 / (s^2 + 2\zeta\omega_0 s + \omega_0^2)$$

Se discretizó con la transformada bilineal, que reemplaza s por $(2/T)(1 - z^{-1})/(1 + z^{-1})$ y conserva la estabilidad del prototipo. Según el teorema de Nyquist, la frecuencia del filtro debe ser menor que la mitad de la frecuencia de muestreo: con $T = 0.05$ s, $f_s = 20$ Hz, así que f_0 debe ser menor que 10 Hz.

Desarrollo

Diseño del filtro

Se eligió una frecuencia de corte de 1 Hz y $\zeta = 0.707$, que corresponde a un filtro Butterworth: responde rápido y con poco sobrepaso. Con $K = 2/T$ y $D = K^2 + 2\zeta\omega_0 K + \omega_0^2$, los coeficientes son:

$$a_0 = a_2 = \omega_0^2/D, \quad a_1 = 2\omega_0^2/D, \quad b_1 = (2\omega_0^2 - 2K^2)/D, \quad b_2 = (K^2 - 2\zeta\omega_0 K + \omega_0^2)/D$$

Tabla 1. Parámetros y coeficientes del filtro.

Parámetro	Valor
Tiempo de muestreo T	0.05 s (f_s = 20 Hz)
Frecuencia de corte f_0	1 Hz (ω_0 = 6.2832 rad/s)
Amortiguamiento ζ	0.707
a_0, a_1, a_2	0.019790, 0.039580, 0.019790
b_1, b_2	−1.564546, 0.643707
Polos	0.7823 ± j0.1782 (|p| = 0.8023)
Ganancia en CD	$(a_0 + a_1 + a_2)/(1 + b_1 + b_2) = 1$

Ambos polos tienen magnitud 0.8023, menor que 1, por lo que el filtro es estable. La ganancia en CD es 1: ante una entrada constante, la salida converge al mismo valor. Como f_0 es mucho menor que f_s, la distorsión de frecuencia de la transformada bilineal es pequeña: el corte digital resultante es de 0.992 Hz. Los coeficientes se calculan en Python al iniciar el programa y se guardan en un arreglo de flotantes $C = [a_0, a_1, a_2, b_1, b_2]$.

Rutina en ensamblador

La rutina *iir* recibe en r0, r1 y r2 las direcciones de los arreglos U = [x[k], x[k−1], x[k−2]], Y = [y[k], y[k−1], y[k−2]] y C. Como pidió el profesor, no contiene ciclos: cada llamada procesa una sola muestra en línea recta. Carga las entradas, las salidas pasadas y los coeficientes en los registros s0 a s9 con *vldr*, calcula los cinco productos con *vmul* y los combina con dos *vadd* y dos *vsub* para obtener y[k] en s10. Después recorre las muestras en memoria con *vstr*: x[k−1] pasa a x[k−2], x[k] pasa a x[k−1], y lo mismo con las salidas. Así, la siguiente llamada ya encuentra el historial actualizado.

En Python, el programa genera una onda cuadrada de 0 a 1 con semiciclos de 20 muestras (1 s), escribe cada muestra en U[0], llama a la rutina e imprime la entrada y la salida en una línea. Con esos dos números por línea, el Plotter de Thonny dibuja las dos curvas. Entre muestras espera T = 0.05 s con *time.sleep*.

Pruebas y resultados

Antes de la gráfica, el programa aplica un escalón unitario durante 100 muestras y compara cada salida del ensamblador con la misma ecuación evaluada en Python (Figura 1).



Figura 1. Respuesta al escalón en la Pico 2: coeficientes, salida del ensamblador contra Python y valor final.

Tabla 2. Respuesta al escalón medida en la Pico 2.

k	Tiempo (s)	y[k]
0	0.00	0.019790
1	0.05	0.090333
2	0.10	0.207751
5	0.25	0.618089
10	0.50	0.997673
20	1.00	1.012292
50	2.50	1.000010
99	4.95	1.000000

La diferencia máxima entre el ensamblador y Python fue 0, y la salida se estabilizó en 1.000000, lo que confirma la ganancia unitaria en CD. Según la simulación en Python de la misma ecuación, el valor máximo es 1.045 en la muestra 14 (0.70 s): un sobrepaso de 4.5 %, muy cercano al 4.3 % que predice la teoría para ζ = 0.707. El tiempo de subida del 10 % al 90 % es de unos 0.30 s, y a partir de 0.95 s la salida queda dentro del 2 % del valor final. La salida apenas oscila antes de estabilizarse, como se espera con este amortiguamiento.

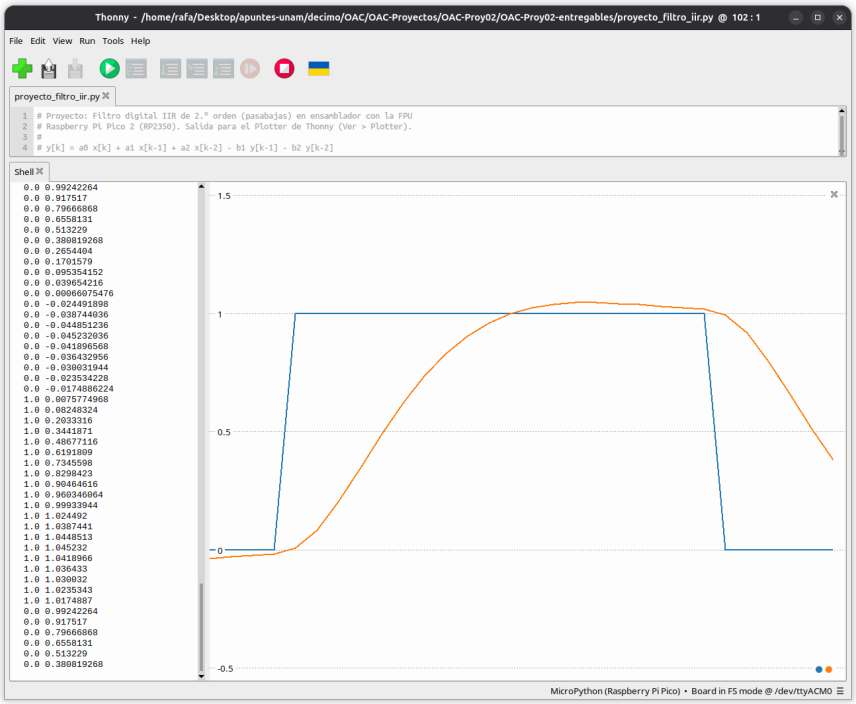


Figura 2. Plotter de Thonny: onda cuadrada de entrada y salida filtrada.

En el Plotter (Figura 2), la entrada cambia de golpe entre 0 y 1 cada segundo, mientras que la salida sube y baja de forma suave, con un sobrepaso pequeño en cada cambio. El filtro pasabajas atenúa las componentes de alta frecuencia que forman los flancos de la onda cuadrada y deja pasar el nivel promedio.

Conclusiones

Se diseñó un filtro pasabajas Butterworth de segundo orden con la transformada bilineal y se implementó en ensamblador con la FPU de la Pico 2. La rutina no tiene ciclos y procesa una muestra por llamada: los cinco productos y cuatro sumas de la ecuación en diferencias, más el recorrido de las muestras pasadas en memoria. La salida del ensamblador coincidió exactamente con el cálculo en Python, y la respuesta al escalón tuvo ganancia unitaria, un sobrepaso de 4.5 % y un tiempo de establecimiento menor a 1

s, de acuerdo con el diseño. El proyecto reúne la operación de multiplicar y acumular de la tarea de producto punto con el muestreo periódico de una señal, que es la base del procesamiento digital de señales en microcontroladores.

Bibliografía

Arm Limited. (2023). *Armv8-M architecture reference manual* (DDI0553). Arm Limited.

Arredondo Garza, J. A. de J. (2026). *Organización y Arquitectura de Computadoras: clases del semestre 2027-1* [Exposiciones en clase]. Facultad de Ingeniería, Universidad Nacional Autónoma de México.

MicroPython. (s. f.). *Inline assembler for Thumb2 architectures*. Recuperado el 19 de octubre de 2026 de https://docs.micropython.org/en/latest/reference/asm_thumb2_index.html

Ogata, K. (2010). *Ingeniería de control moderna* (5.ª ed.). Pearson Educación.

Oppenheim, A. V., & Schafer, R. W. (2010). *Discrete-time signal processing* (3.ª ed.). Pearson.

Raspberry Pi Ltd. (2025). *RP2350 datasheet: A microcontroller by Raspberry Pi*. https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf

Anexo: código fuente (proyecto_filtro_iir.py)

```
# Proyecto: Filtro digital IIR de 2.º orden (pasabajas) en ensamblador con la FPU
# Raspberry Pi Pico 2 (RP2350). Salida para el Plotter de Thonny (Ver > Plotter).
#
# y[k] = a0 x[k] + a1 x[k-1] + a2 x[k-2] - b1 y[k-1] - b2 y[k-2]
# Diseño: H(s) = w0^2 / (s^2 + 2 z w0 s + w0^2), discretizado por transformada
# bilineal s = (2/Ts)(1 - z^-1)/(1 + z^-1).
from array import array
import math
import time

TS = 0.05            # tiempo de muestreo [s]  -> fs = 20 Hz, Nyquist = 10 Hz
F0 = 1.0             # frecuencia de corte [Hz] (debe ser < 10 Hz)
ZETA = 0.707         # amortiguamiento (0.707 = Butterworth)
MEDIO_PERIODO = 20   # muestras por semiciclo de la onda cuadrada (1 s)
RUIDO = False        # True: agrega ruido a la entrada para ver el filtrado

assert F0 < 1 / (2 * TS), "F0 debe ser menor que la frecuencia de Nyquist"

def coeficientes(ts, f0, zeta):
    w0 = 2 * math.pi * f0
    k = 2 / ts
    d = k * k + 2 * zeta * w0 * k + w0 * w0
    a0 = w0 * w0 / d
    a1 = 2 * w0 * w0 / d
    a2 = w0 * w0 / d
    b1 = (2 * w0 * w0 - 2 * k * k) / d
    b2 = (k * k - 2 * zeta * w0 * k + w0 * w0) / d
    return a0, a1, a2, b1, b2

C = array('f', coeficientes(TS, F0, ZETA))   # a0, a1, a2, b1, b2
U = array('f', [0.0, 0.0, 0.0])              # x[k], x[k-1], x[k-2]
Y = array('f', [0.0, 0.0, 0.0])              # y[k], y[k-1], y[k-2]

# r0 -> U, r1 -> Y, r2 -> C.  Sin ciclos: una muestra por llamada.
@micropython.asm_thumb
def iir(r0, r1, r2):
    vldr(s0, [r0, 0])     # x[k]
    vldr(s1, [r0, 4])     # x[k-1]
    vldr(s2, [r0, 8])     # x[k-2]
    vldr(s3, [r1, 4])     # y[k-1]
    vldr(s4, [r1, 8])     # y[k-2]
    vldr(s5, [r2, 0])     # a0
    vldr(s6, [r2, 4])     # a1
    vldr(s7, [r2, 8])     # a2
    vldr(s8, [r2, 12])    # b1
    vldr(s9, [r2, 16])    # b2
    vmul(s10, s0, s5)     # a0 x[k]
    vmul(s11, s1, s6)     # a1 x[k-1]
    vmul(s12, s2, s7)     # a2 x[k-2]
    vmul(s13, s3, s8)     # b1 y[k-1]
    vmul(s14, s4, s9)     # b2 y[k-2]
    vadd(s10, s10, s11)
```

```
    vadd(s10, s10, s12)
    vsub(s10, s10, s13)
    vsub(s10, s10, s14)   # s10 = y[k]
    vstr(s10, [r1, 0])    # y[k]
    vstr(s1, [r0, 8])     # x[k-2] <- x[k-1]
    vstr(s0, [r0, 4])     # x[k-1] <- x[k]
    vstr(s3, [r1, 8])     # y[k-2] <- y[k-1]
    vstr(s10, [r1, 4])    # y[k-1] <- y[k]

def reiniciar():
    for i in range(3):
        U[i] = 0.0
        Y[i] = 0.0

def filtrar(x):
    U[0] = x              # escribe la muestra en el arreglo (no reemplazar U)
    iir(U, Y, C)
    return Y[0]

# ---------- Prueba: respuesta al escalón, asm vs Python ----------
print("Coeficientes  a0={:.6f} a1={:.6f} a2={:.6f} b1={:.6f} b2={:.6f}".format(*C))
reiniciar()
xr = [0.0, 0.0]; yr = [0.0, 0.0]
a0, a1, a2, b1, b2 = C
err_max = 0.0
for k in range(100):
    y = filtrar(1.0)
    y_ref = a0 * 1.0 + a1 * xr[0] + a2 * xr[1] - b1 * yr[0] - b2 * yr[1]
    xr = [1.0, xr[0]]; yr = [y_ref, yr[0]]
    err_max = max(err_max, abs(y - y_ref))
    if k in (0, 1, 2, 5, 10, 20, 50, 99):
        print("k={:>3}  y_asm={:.6f}  y_python={:.6f}".format(k, y, y_ref))
print("Error maximo asm vs Python: {:.2e}".format(err_max))
print("Valor final (ganancia DC esperada = 1.0): {:.6f}".format(y))
time.sleep(3)

# ---------- Demostración: onda cuadrada para el Plotter de Thonny ----------
if RUIDO:
    import random
reiniciar()
k = 0
while True:
    x = 1.0 if (k // MEDIO_PERIODO) % 2 == 0 else 0.0
    if RUIDO:
        x += (random.random() - 0.5) * 0.4
    y = filtrar(x)
    print(x, y)           # dos números por línea: entrada y salida
    time.sleep(TS)
    k += 1
```